0235. Lowest Common Ancestor of a BST

ory: Trees | LeetCode: https://leetcode.com/problems/lowest-common-ancestor

Problem

- In a BST, return the lowest node that has both p and q as descendants (a node is a descendant of itself).

Intuition

- BST ordering tells us where p and q live. If both are smaller than the current node, the LCA is in the left subtree; if both are larger, the right.
- Otherwise they split here (or one equals the node), so this node is the LCA.

Brute Force

- Generic binary-tree LCA (ignoring BST order): O(n).

Optimal Approach

- Cur = root.
- If p and q < cur, go left; if both > cur, go right; else return cur.

Worked Example

```
BST [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4.

cur=6: 2<6 and 4<6 -> left
cur=2: 2 is not < 2 -> split point -> LCA = 2
```

Java Solution

```java
public class LowestCommonAncestorBST {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode cur = root;
        while (cur != null) {
            if (p.val < cur.val && q.val < cur.val) cur = cur.left;
            else if (p.val > cur.val && q.val > cur.val) cur = cur.right;
            else return cur;
        }
        return null;
    }
}
```

Complexity

- Time: O(h). Space: O(1) iterative.

Pitfalls

- The "equal" case must return the current node.